In [ ]:
```
import marimo as mo
import cvxpy as cp
import numpy as np
import mosek
from time import perf_counter
from itertools import product
```

# Resolução de uma versão limitada do CVP

O objetivo é encontrar um ponto de um reticulado que esteja o mais próximo possível de um alvo, usando a distância euclidiana.

Neste trabalho, as linhas de uma matriz $G$ são $k$ vetores linearmente independentes, cada um com $m$ componentes geradas uniformemente em $[0, 1]$. O alvo é construído como $t = G^T u$, com coeficientes reais $u_i$ uniformes em $[0, \ell]$.

Procuram-se coeficientes **inteiros** $z_i$, entre $0$ e $\ell$, para formar o ponto $x = G^T z$ mais próximo de $t$.

- **k**: número de vetores da base e de coeficientes inteiros a escolher.
- **m = 2k**: número de componentes de cada vetor.
- **ell**: limite superior dos coeficientes inteiros.

## Porque usei MOSEK em vez de Clarabel

O problema tem duas características: uma restrição cónica para representar a distância e variáveis inteiras para representar os coeficientes do reticulado.

O **MOSEK suporta ambas diretamente**, através do CVXPY. O **Clarabel resolve problemas cónicos contínuos**, mas não trata diretamente variáveis inteiras. Para o usar neste exercício, seria necessário implementar também um método de pesquisa inteira, como *branch-and-bound*.

A escolha do MOSEK permite manter o código simples e resolver a formulação inteira do enunciado. Requer uma licença válida, por exemplo académica.

Retirar a integralidade não resolve o exercício: nesse caso, $z = u$ daria distância zero. Arredondar $u$ também não garante a melhor solução inteira.

Fontes: solvers do CVXPY e otimização inteira no MOSEK.

## Formulação matemática

Introduz-se uma variável $r$ que limita superiormente a distância ao alvo:

$$\begin{aligned} \min_{z,r} \quad & r \\ \text{sujeito a} \quad & \|G^T z - t\|_2 \le r, \\ & z \in \mathbb{Z}^k, \quad 0 \le z_i \le \ell, \quad r \ge 0. \end{aligned}$$

Em CVXPY, a restrição da distância escreve-se com um cone de segunda ordem:

```
z = cp.Variable(k, integer=True)
r = cp.Variable(nonneg=True)
restricoes = [cp.SOC(r, G.T @ z - t), z >= 0, z <= ell]
problema = cp.Problem(cp.Minimize(r), restricoes)
problema.solve(solver=cp.MOSEK)
```

## Código do trabalho

Todo o código está neste documento. As funções seguintes geram os dados, verificam a solução e permitem confirmar os casos pequenos por pesquisa exaustiva. A construção e a resolução do modelo aparecem depois dos controlos.

In [ ]:
```
def validar_parametros(k, m, ell):
    for valor in (k, m, ell):
        if isinstance(valor, (bool, np.bool_)) or not isinstance(valor,
(int, np.integer)) or valor <= 0:
            raise ValueError("k, m e ell devem ser inteiros positivos.")
    if k > m:
        raise ValueError("O número de vetores não pode exceder a
dimensão.")

def gerar_base(k, m, rng):
    validar_parametros(k, m, 1)
    while True:
        G = rng.uniform(0, 1, size=(k, m))
        if np.linalg.matrix_rank(G) == k:
            return G

def gerar_alvo(G, ell, rng):
    validar_parametros(G.shape[0], G.shape[1], ell)
    u = rng.uniform(0, ell, size=G.shape[0])
    t = G.T @ u
    return u, t

mo.show_code()
```

In [ ]:
```
def verificar_solucao(G, t, ell, z_valor, r_valor, atol=1e-5, rtol=1e-7):
    if z_valor is None or r_valor is None:
        raise ValueError("O solver não devolveu valores para a solução.")
    if not np.all(np.isfinite(z_valor)) or not np.isfinite(r_valor):
        raise ValueError("A solução contém valores não finitos.")

    z_inteiro = np.rint(z_valor)
    if not np.allclose(z_valor, z_inteiro, atol=atol, rtol=rtol):
```

```
        raise ValueError("Os coeficientes não são inteiros dentro da
tolerância.")
    if np.any(z_inteiro < 0) or np.any(z_inteiro > ell):
        raise ValueError("Os coeficientes não respeitam os limites.")

    # Verificar nas coordenadas originais, antes e depois do
arredondamento.
    residuo = np.linalg.norm(G.T @ z_valor - t)
    z_inteiro = z_inteiro.astype(int)
    x = G.T @ z_inteiro
    distancia = np.linalg.norm(x - t)
    margem = atol + rtol * max(abs(r_valor), residuo, distancia)
    if r_valor < -atol or max(residuo, distancia) > r_valor + margem:
        raise ValueError("A solução não respeita a restrição cónica.")
    return z_inteiro, x, distancia

mo.show_code()
```

In [ ]:
```
def resolver_exaustivo(G, t, ell):
    k = G.shape[0]
    if (ell + 1) ** k > 100_000:
        raise ValueError("A pesquisa exaustiva destina-se apenas a casos
pequenos.")

    melhor_z = None
    melhor_distancia = np.inf
    for coeficientes in product(range(ell + 1), repeat=k):
        z = np.array(coeficientes)
        distancia = np.linalg.norm(G.T @ z - t)
        if distancia < melhor_distancia:
            melhor_z = z
            melhor_distancia = distancia
    return melhor_z, melhor_distancia

mo.show_code()
```

## Experimentar outros valores

Escolhe **k** e **ell** e carrega em **Resolver**. A semente fica em 42 para permitir repetir a mesma experiência. O caso k=5 é uma validação pequena; os valores do enunciado são 30, 40 e 50 e podem demorar vários minutos.

O resultado só é atualizado depois de carregares no botão e de a resolução terminar.

In [ ]:
```
escolhas = mo.ui.dictionary({
    "k": mo.ui.dropdown([5, 30, 40, 50], value=5, label="k"),
    "ell": mo.ui.slider(start=1, stop=5, step=1, value=1, label="ell",
show_value=True),
}).form(submit_button_label="Resolver", show_clear_button=False)
escolhas
```

In [ ]:
```
mo.stop(escolhas.value is None, mo.md("Escolhe os parâmetros e carrega em
**Resolver**."))
```

```
k = escolhas.value["k"]
ell = escolhas.value["ell"]
m = 2 * k
rng = np.random.default_rng(42)
G = gerar_base(k, m, rng)
u, t = gerar_alvo(G, ell, rng)
mo.show_code()
```

In [ ]:
```
# O cronómetro inclui a construção do modelo e a chamada completa a
solve().
inicio = perf_counter()
z = cp.Variable(k, integer=True)
r = cp.Variable(nonneg=True)
restricoes = [cp.SOC(r, G.T @ z - t), z >= 0, z <= ell]
problema = cp.Problem(cp.Minimize(r), restricoes)

erro_resolucao = None
try:
    problema.solve(solver=cp.MOSEK)
except (cp.error.SolverError, mosek.Error) as erro:
    erro_resolucao = str(erro)
tempo = perf_counter() - inicio
mo.show_code()
```

In [ ]:
```
mo.stop(
    erro_resolucao is not None,
    mo.callout(f"Não foi possível resolver com MOSEK: {erro_resolucao}",
kind="danger"),
)
mo.stop(
    z.value is None or r.value is None,
    mo.callout(f"Estado: {problema.status}. Sem solução devolvida;
optimalidade não confirmada.", kind="warn"),
)

erro_validacao = None
try:
    z_inteiro, _, distancia = verificar_solucao(G, t, ell, z.value,
r.value)
except ValueError as erro:
    erro_validacao = str(erro)
mo.stop(erro_validacao is not None, mo.callout(f"A validação falhou:
{erro_validacao}", kind="danger"))

conclusao = "Ótima às tolerâncias do solver." if problema.status ==
cp.OPTIMAL else "Admissível; optimalidade não confirmada pelo solver."
verificacao = "Sem pesquisa exaustiva: o número de combinações é demasiado
elevado."
if (ell + 1) ** k <= 100_000:
    _, distancia_exaustiva = resolver_exaustivo(G, t, ell)
    coincide = np.isclose(distancia, distancia_exaustiva, atol=1e-5,
rtol=1e-7)
    verificacao = "A distância coincide com o ótimo da pesquisa exaustiva."
if coincide else "A distância não coincide com o ótimo da pesquisa
exaustiva."
```

```
    mo.stop(
        problema.status == cp.OPTIMAL and not coincide,
        mo.callout("O ótimo reportado não passou a confirmação por pesquisa
exaustiva.", kind="danger"),
    )

mo.md(f"""
## Resultado para k={k} e ell={ell}

| Medida | Resultado |
| --- | --- |
| Dimensão de G | {k} × {m} |
| Estado do solver | {problema.status} |
| Distância ao alvo | {distancia:.10f} |
| Preparação e resolução | {tempo:.3f} s |

**Coeficientes encontrados:** `{z_inteiro.tolist()}`

**Conclusão:** {conclusao}

{verificacao}
""")
```

## Como interpretar os resultados

A distância apresentada é calculada novamente com `G.T @ z_inteiro - t`. Antes de mostrar o resultado, o programa verifica a integralidade, os limites e a restrição cónica, incluindo a solução depois do arredondamento.

O tempo inclui preparação, compilação do CVXPY, resolução e recuperação do resultado. Não inclui geração dos dados nem pesquisa exaustiva. As tolerâncias de optimalidade do MOSEK mantêm os valores por defeito.

Com a mesma semente e k fixo, mudar ell mantém G mas escala u e o alvo t. Por isso, um ell maior não implica necessariamente mais tempo. Ao mudar k, também muda m e a instância gerada. O número de combinações é $(\ell + 1)^k$, mas o solver não precisa de as enumerar todas.